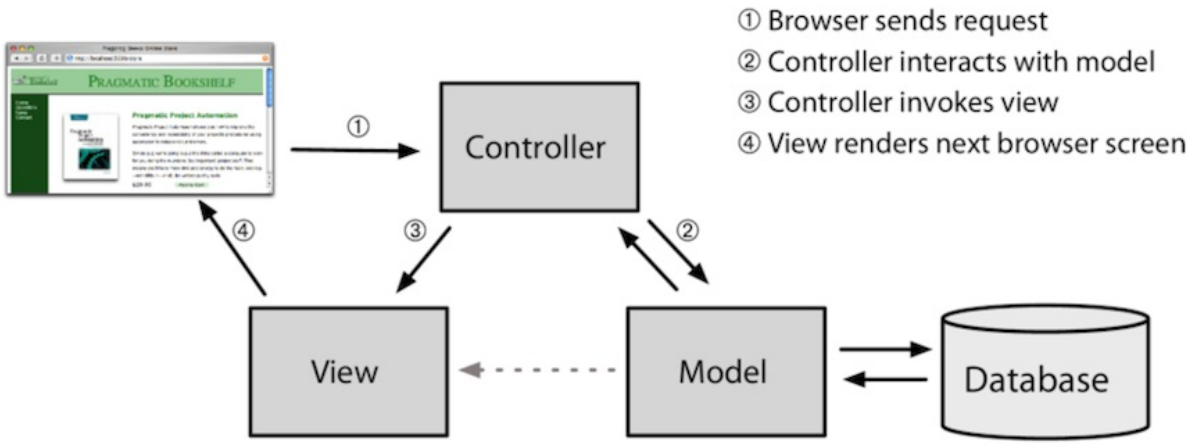
following figure.



The MVC architecture was originally intended for conventional GUI applications, where developers found that the separation of concerns led to far less coupling, which in turn made the code easier to write and maintain. Each concept or action was expressed in a single, well-known place. Using MVC was like constructing a skyscraper with the girders already in place—it was a lot easier to hang the rest of the pieces with a structure already there. During the development of our application, we'll make heavy use of Rails' ability to generate *scaffolding* for our application.

Ruby on Rails is an MVC framework too. Rails enforces a structure for your application: you develop models, views, and controllers as separate chunks of functionality, and it knits them together as your program executes. One of the joys of Rails is that this knitting process is based on the use of intelligent defaults so that you typically don't need to write any external configuration metadata to make it all work. This is an example of the Rails philosophy of favoring convention over configuration.

In a Rails application, an incoming request is first sent to a router, which works out where in the application the request should be sent and how the request should be parsed. Ultimately, this phase identifies a particular method (called an *action* in Rails parlance) somewhere in the controller code. The action might look at data in the request, it might interact with the model, and it might cause other actions to be invoked. Eventually the action prepares information for the view, which renders something to the user.